

TJSC 2026 - Analista de Sistemas

Arquitetura de Sistemas e Integração

Aula 4

Integração de Sistemas, APIs RESTful, JSON, Webhooks e APIs Reversas

Campo	Descrição
Disciplina	Arquitetura de Sistemas e Integração - 10 questões no bloco de conhecimentos específicos
Aula	Aula 4 da trilha nova
Itens do edital cobertos	1.5 Integração de sistemas; 1.6 APIs RESTful e formato JSON; 1.7 Webhooks e APIs reversas
Foco de prova	Diferenciar integração síncrona/assíncrona, REST/HTTP/JSON, webhooks/polling e API direta/API reversa
Padrão	Teoria objetiva e aprofundada, exemplos corporativos, pegadinhas FGV, 20 questões autorais, gabarito separado e comentários completos

Material autoral para preparação objetiva - banca FGV

1. Identificação da aula e recorte do edital

Esta aula corresponde à Aula 4 da trilha de Arquitetura de Sistemas e Integração. O recorte é intencionalmente concentrado em integração entre sistemas e contratos HTTP: integração de sistemas, APIs RESTful, JSON, webhooks e APIs reversas. O objetivo é transformar experiência prática com APIs, Java/Quarkus, Angular/TypeScript e ambiente corporativo em acerto de prova FGV.

O edital vigente do TJSC para Analista de Sistemas inclui expressamente, na disciplina Arquitetura de Sistemas e Integração: 1.5 Integração de sistemas; 1.6 APIs RESTful e formato JSON; 1.7 Webhooks e APIs reversas. Esta aula não entra em Spring Cloud, API Gateway, Service Discovery ou mensageria RabbitMQ em profundidade, porque esses tópicos serão tratados nas aulas próprias. Aqui eles aparecem apenas quando necessários para delimitar conceitos próximos.

Resultado esperado
Ao final, você deve conseguir resolver questões que peçam: identificar o estilo de integração mais adequado; distinguir REST de HTTP e de JSON; reconhecer método HTTP correto; diferenciar payload, endpoint, request e response; distinguir webhook de polling; entender API reversa como padrão de callback/endpoint exposto pelo consumidor; e detectar alternativas que confundem contrato, protocolo, formato de dados e arquitetura.

2. Como a FGV tende a cobrar este bloco

A FGV costuma trabalhar com enunciados técnicos contextualizados: órgão público, tribunal, sistema corporativo, integração entre plataformas, fluxo de dados, decisão de arquitetura ou trecho de API. A banca raramente pergunta apenas o verbete; ela oferece alternativas com termos verdadeiros em contextos errados.

Em prova correlata de TI, a FGV já cobrou API REST com foco em recursos, métodos GET/POST/PUT/DELETE e statelessness. A alternativa correta explorava a ideia de que, em REST, a comunicação é sem estado entre requisições. Esse padrão é importante: a banca gosta de trocar REST por SOAP, XML, WSDL, mensageria ou estado de sessão.

Padrão provável	Como aparece	Risco de erro
Conceito vizinho	REST, HTTP, JSON, SOAP, RPC, webhook e mensageria aparecem no mesmo enunciado.	Escolher um termo verdadeiro, mas que não resolve o caso.
Cenário corporativo	Angular chama API Java; sistema externo envia evento; legado exige adaptação.	Responder com ferramenta em vez de princípio arquitetural.
Método HTTP	Criação, leitura, atualização, exclusão, idempotência e segurança.	Confundir POST com PUT, idempotente com seguro.
Falha e retry	Webhook é reenviado; consumidor processa evento duas vezes.	Ignorar idempotência e identificador de evento.
Contrato de integração	Payload JSON, schema, versionamento e status code.	Confundir payload com endpoint ou JSON com protocolo.

3. Termos e siglas essenciais

Termo/sigla	Expansão ou definição	Como guardar para a prova
API	Application Programming Interface - Interface de Programação de Aplicações.	Contrato pelo qual um sistema expõe funcionalidades ou dados para outro.
Endpoint	Ponto de acesso de uma API, normalmente uma URI associada a uma operação.	Ex.: GET /processos/123. Não é o sistema inteiro; é uma porta de entrada específica.
Request	Requisição enviada pelo cliente ao servidor.	Tem método, URI, headers e eventualmente corpo.
Response	Resposta devolvida pelo servidor ao cliente.	Tem status code, headers e eventualmente corpo.
Payload	Carga útil de dados transportada na mensagem.	Em APIs, costuma ser o corpo JSON de request ou response.
HTTP	Hypertext Transfer Protocol.	Protocolo de aplicação de

		request/response. É stateless por natureza.
TLS	Transport Layer Security.	Camada de segurança usada no HTTPS para proteger comunicação em trânsito.
DNS	Domain Name System.	Resolve nomes como api.tjsc.jus.br para endereços IP.
REST	Representational State Transfer.	Estilo arquitetural; não é protocolo e não exige JSON.
JSON	JavaScript Object Notation.	Formato textual leve para intercâmbio de dados. Não é banco, protocolo nem API.
URI/URL	Uniform Resource Identifier / Uniform Resource Locator.	Identifica/localiza recurso. Em REST, deve representar recursos, não ações internas.
Webhook	Callback HTTP orientado a evento.	O provedor chama um endpoint do consumidor quando algo acontece.
API reversa	Uso em que o consumidor/parceiro expõe endpoint para ser chamado pelo provedor da integração.	Pense em callback/webhook: o fluxo de chamada se inverte em relação à API tradicional.
Polling	Consulta periódica a uma API para verificar se há novidade.	O cliente pergunta várias vezes; diferente de webhook, em que o provedor avisa.
DTO	Data Transfer Object.	Objeto de transferência que representa contrato de entrada/saída, evitando expor entidade interna.

4. Integração de sistemas

Integração de sistemas é o conjunto de decisões técnicas e arquiteturais usadas para fazer sistemas distintos trocarem dados, comandos ou eventos de forma controlada. No seu mundo corporativo, isso pode envolver Angular consumindo uma API Java/Quarkus, um backend chamando outro serviço, um sistema legado recebendo arquivo em lote, uma plataforma externa enviando webhook, ou uma fila intermediando eventos negociais.

Para prova, o mais importante não é decorar produtos, mas reconhecer o tipo de acoplamento, o sentido do fluxo, o tempo de resposta esperado e o contrato de dados. A FGV pode perguntar: quando usar API síncrona? Quando evitar acesso direto ao banco? Quando preferir webhook a polling? O que torna uma integração frágil?

Estilo de integração	Característica	Vantagem	Risco/limite
API síncrona HTTP	Cliente chama endpoint e espera response.	Resposta imediata e contrato claro.	Acoplamento temporal: se o servidor cair ou demorar, o cliente sofre.
Webhook/API reversa	Provedor chama endpoint do consumidor quando evento ocorre.	Evita polling e melhora reatividade.	Exige endpoint público/roteável, segurança, retry e idempotência.
Polling	Cliente consulta periodicamente uma API.	Simples quando o provedor não envia eventos.	Ineficiente, gera atraso e carga desnecessária.
Batch/arquivo	Troca em lote por arquivo, agenda ou carga periódica.	Bom para volume e sistemas legados.	Baixa tempestividade e maior risco de inconsistência temporária.
Acesso direto ao banco	Sistema externo consulta tabelas de outro sistema.	Pode parecer rápido no curto prazo.	Acoplamento forte ao schema, risco de segurança e quebra de encapsulamento.
Mensageria/eventos	Produtor publica mensagem; consumidor processa depois.	Desacoplamento temporal e resiliência.	Requer broker, governança de eventos e consistência eventual.

Regra de prova

Integração boa não é a que simplesmente faz dois sistemas se falarem. Integração boa preserva contrato, segurança, evolução, observabilidade, tratamento de falha e baixo acoplamento. Se a alternativa propõe acessar diretamente o banco do outro sistema para ganhar velocidade, desconfie.

5. APIs RESTful

REST - Representational State Transfer - **é um estilo arquitetural para sistemas distribuídos**. Em provas de TI, costuma aparecer implementado sobre HTTP, mas REST não é o próprio HTTP. Também não é sinônimo de JSON. Uma API pode usar HTTP e JSON sem ser bem RESTful, se ela expõe apenas ações proceduralmente, ignora recursos, usa métodos sem semântica coerente e mantém estado de conversação no servidor.

5.1 Recursos, representações e URI

Em REST, o desenho deve partir de recursos. Um recurso é algo relevante no domínio que pode ser identificado e manipulado por representações. Exemplo: processo, parte, documento, movimentação, protocolo, usuário, unidade. A representação é a forma transferida ao cliente, por exemplo um JSON com os dados de um processo.

Menos aderente a REST	Mais aderente a REST	Motivo
POST /consultarProcesso	GET /processos/{numero}	O recurso é processo; a ação de consultar já está no método GET.
POST /criarDocumento	POST /documentos	POST na coleção sugere criação de novo recurso.
POST /alterarUsuario/10	PUT /usuarios/10 ou PATCH /usuarios/10	URI identifica o recurso; método indica substituição ou alteração parcial.
GET /deletarArquivo?id=5	DELETE /arquivos/5	GET não deve ser usado para operação que altera estado.

5.2 Métodos HTTP e semântica de prova

Método	Uso comum em REST	Seguro?	Idempotente?	Pegadinha FGV
GET	Obter representação de recurso.	Sim	Sim	Não deve alterar estado negocial; logs e cache não mudam a semântica pretendida.
HEAD	Obter metadados como em GET, sem corpo.	Sim	Sim	Útil para verificar existência ou headers.
POST	Criar recurso em uma coleção ou submeter processamento.	Não	Geralmente não	Pode criar recurso novo a cada chamada. Não é sinônimo de qualquer operação.
PUT	Substituir o recurso identificado pela URI; pode criar se não existir, conforme contrato.	Não	Sim	Cliente conhece a URI do recurso. Repetir tende ao mesmo estado final.
PATCH	Atualizar parcialmente um recurso.	Não	Não necessariamente	Depende do formato e da operação. Não é sempre idempotente.
DELETE	Remover recurso identificado.	Não	Sim	Repetir DELETE deve manter o recurso removido, embora a resposta possa mudar.

5.3 Status codes que mais rendem alternativa

Código	Uso típico	Erro comum
--------	------------	------------

200 OK	Resposta bem-sucedida com corpo.	Usar 200 para tudo, inclusive criação sem indicar localização.
201 Created	Recurso criado; costuma combinar com header Location.	Trocar por 204 quando há criação.
202 Accepted	Solicitação aceita para processamento assíncrono.	Achar que 202 significa processamento concluído.
204 No Content	Sucesso sem corpo de resposta.	Esperar corpo JSON obrigatório.
400 Bad Request	Requisição malformada ou inválida sintaticamente.	Confundir com autenticação.
401 Unauthorized	Falta autenticação válida.	Confundir com 403.
403 Forbidden	Cliente autenticado, mas sem permissão.	Confundir com recurso inexistente.
404 Not Found	Recurso não encontrado.	Usar para qualquer erro de negócio.
409 Conflict	Conflito com estado atual do recurso.	Não usar em duplicidade/conflito de versão.
422 Unprocessable Content/Entity	Semântica inválida, mesmo com sintaxe compreensível, quando adotado pelo contrato.	Tratar como obrigatório em toda API.
500 Internal Server Error	Falha interna inesperada.	Usar para erro validável do cliente.
503 Service Unavailable	Serviço indisponível temporariamente.	Confundir com 500 genérico.

5.4 REST x HTTP x JSON x SOAP x RPC

Conceito	O que é	Não confundir com
HTTP	Protocolo de aplicação request/response.	REST; HTTP pode transportar APIs REST, RPC, páginas HTML, arquivos etc.
REST	Estilo arquitetural orientado a recursos, interface uniforme e stateless.	JSON; REST pode usar XML, JSON ou outro formato.
JSON	Formato de intercâmbio de dados.	API; JSON sozinho não define autenticação, endpoint nem método.
SOAP	Protocolo de serviços web com envelope XML e contratos típicos via WSDL.	REST; SOAP é mais formal e orientado a operações.
RPC	Chamada remota com foco em executar procedimento/ação.	REST; RPC tende a expor operações como chamarMetodo().
GraphQL	Linguagem/estilo de consulta em que o cliente especifica a forma dos dados desejados.	REST; não é sinônimo de JSON nem substitui todo cenário.

6. Formato JSON

JSON - JavaScript Object Notation - é formato textual, leve e independente de linguagem para intercâmbio de dados. A especificação descreve tipos primitivos como string, number, boolean e null, e tipos estruturados como object e array. Em APIs, JSON aparece como payload, não como protocolo de transporte.

Pegadinha central

Uma API RESTful não é RESTful porque usa JSON. Ela é RESTful por causa de suas restrições arquiteturais e do desenho orientado a recursos. JSON é apenas uma representação possível dos dados transferidos.

6.1 Exemplo de payload JSON

```
{
  "numeroProcesso": "0001234-56.2026.8.24.0000",
  "classe": "Procedimento Administrativo",
  "situacao": "EM_TRAMITACAO",
```

```

"partes": [
  {"tipo": "REQUERENTE", "nome": "Maria Souza"},
  {"tipo": "REQUERIDO", "nome": "Estado de Santa Catarina"}
],
"sigiloso": false
}

```

Elemento JSON	No exemplo	Leitura de prova
Objeto	{...}	Coleção de pares nome/valor.
Array	partes: [...]	Sequência ordenada de valores.
String	"EM_TRAMITACAO"	Texto entre aspas duplas.
Boolean	false	Valor lógico true/false, sem aspas.
null	campo: null	Ausência explícita de valor, diferente de string vazia.
Number	valor: 10.5	Número sem aspas; cuidado com precisão monetária.

6.2 Cuidados de contrato JSON

Em integrações reais, o payload precisa de contrato. Isso pode ser descrito por OpenAPI, JSON Schema, documentação de endpoint, exemplos versionados e testes de contrato. A FGV pode cobrar a diferença entre dado transferido e estrutura interna: um DTO JSON não deve obrigatoriamente ser igual a uma entidade JPA ou a uma tabela de banco.

Decisão	Boa prática	Erro comum
Datas	Definir formato, fuso horário e semântica.	Enviar data local ambígua sem timezone quando o domínio exige precisão.
Valores monetários	Definir escala, moeda e arredondamento.	Usar number sem critério e gerar divergência de precisão.
Enums	Documentar valores possíveis e evolução.	Trocar texto livre por enum sem versionar consumidores.
Campos opcionais	Distinguir ausente, null e valor vazio.	Achar que null, vazio e não enviado são sempre equivalentes.
Compatibilidade	Preferir mudanças aditivas para não quebrar clientes.	Renomear/remover campo em contrato público sem versão.

7. Webhooks

Webhook é um mecanismo de integração orientado a evento em que um sistema provedor envia uma chamada HTTP para um endpoint configurado pelo sistema consumidor quando determinado evento ocorre. Em vez de o consumidor perguntar periodicamente se algo mudou, o provedor avisa.

7.1 Fluxo básico

1. Sistema consumidor cadastra uma URL de callback: <https://cliente.exemplo.gov.br/webhooks/processos>
2. Sistema provedor registra que deve notificar eventos como processo.movimentado.
3. Quando o evento ocorre, o provedor envia POST para a URL cadastrada.
4. O consumidor valida assinatura/autenticidade, registra o evento e responde 2xx rapidamente.
5. Se a resposta falha ou o tempo expira, o provedor pode tentar reenviar.

Webhook	Polling
Push: o provedor envia evento ao consumidor.	Pull: o consumidor consulta periodicamente.
Melhor para notificação quase em tempo real.	Pode ser aceitável quando atraso é tolerável.
Exige endpoint acessível, autenticação, assinatura e retry.	Exige agendamento e controle de carga de consulta.
Pode duplicar eventos em caso de retry; consumidor deve ser idempotente.	Pode perder janela ou gerar carga excessiva se intervalo for ruim.
Não substitui necessariamente uma API de consulta detalhada.	Pode complementar APIs quando não há mecanismo de notificação.

7.2 Segurança, retry e idempotência em webhooks

A prova pode tentar vender webhook como mecanismo mágico de integração em tempo real. Não é. Webhooks precisam de validação de origem, TLS, assinatura ou token, tratamento de replay, timeouts, logs, correlation id, controle de duplicidade e idempotência. O consumidor deve estar preparado para receber o mesmo evento mais de uma vez.

Problema	Medida adequada	Pegadinha
Origem falsa	Assinatura HMAC, token, mTLS ou mecanismo equivalente.	Confiar apenas no nome do domínio de origem.
Replay attack	Timestamp, nonce, janela de validade e assinatura.	Aceitar indefinidamente a mesma mensagem.
Retry duplicado	Idempotência por eventId ou chave de negócio.	Processar pagamento/evento duas vezes.
Processamento demorado	Responder rápido e processar em fila interna.	Manter a chamada aberta até terminar toda regra pesada.
Falha transitória	Retentativas com backoff e observabilidade.	Reenviar sem limite ou sem rastreabilidade.

8. APIs reversas

No contexto de integração, **API reversa não deve ser lida como um protocolo formal universal**. O termo costuma aparecer para indicar a inversão do sentido tradicional da chamada: em vez de o consumidor chamar a API do provedor, o consumidor expõe uma API/endpoint que será chamado pelo provedor, frequentemente como callback ou webhook. Por isso, a expressão aparece próxima de webhooks.

Definição segura para prova			
API reversa: padrão de integração em que o consumidor/parceiro precisa disponibilizar um endpoint seguindo contrato definido para que o provedor envie chamadas, notificações ou eventos. É próxima de callback/webhook. Não é sinônimo de API Gateway, reverse proxy ou REST em si.			
Situação	Quem expõe endpoint?	Quem inicia a chamada?	Nome provável
Angular consulta processos	TJSC/API backend	Cliente Angular	API tradicional/REST
Parceiro cadastra URL para receber evento	Parceiro/consumidor	TJSC/provedor do evento	Webhook/API reversa/callback
Sistema consulta status a cada 5 minutos	TJSC/API backend	Parceiro/cliente	Polling
Gateway roteia chamadas de clientes para serviços internos	Gateway	Cliente externo	API Gateway/reverse proxy, não API reversa no sentido de webhook

9. Exemplos práticos ligados a Angular, Java/Quarkus e integração corporativa

9.1 Consulta síncrona por API REST

Cenário: tela Angular do TJSC consulta dados de processo administrativo. O Angular chama GET /api/processos/{numero}; o Resource/Controller Java/Quarkus valida autenticação e autorização, chama service, consulta repositório e retorna DTO JSON. É integração síncrona, cliente-servidor e orientada a recurso.

9.2 Criação com idempotência

Cenário: sistema externo envia solicitação de protocolo. POST /protocolos cria novo protocolo. Para evitar duplicidade em reenvio por falha de rede, o contrato pode exigir Idempotency-Key no header ou chave externa no payload. Assim, duas tentativas iguais não criam dois protocolos.

9.3 Evento por webhook/API reversa

Cenário: sistema parceiro não quer consultar o TJSC a cada minuto. Ele cadastra uma URL de callback. Quando um processo muda de fase, o provedor envia POST para /webhooks/processos com eventId, tipo, dataHora, numeroProcesso e link de consulta. O consumidor responde 2xx após validar e registrar o evento, processando detalhes depois.

9.4 Integração com legado

Cenário: sistema legado não expõe endpoint HTTP moderno. Uma camada adaptadora pode transformar arquivo ou protocolo antigo em API ou evento padronizado. A decisão relevante é não contaminar todos os consumidores com detalhes do legado; o adaptador isola formato, mapeamento e restrições.

10. Trade-offs que a FGV pode explorar

Decisão	Quando faz sentido	Quando vira problema
REST síncrono	Consulta ou comando que precisa de resposta imediata.	Operações longas, dependência forte e baixa tolerância a falha.
Webhook/API reversa	Notificação de eventos e redução de polling.	Consumidor sem endpoint seguro/estável ou sem idempotência.
Polling	Provedor não suporta webhook; atraso é tolerável.	Necessidade de tempo real ou volume alto de consultas vazias.
JSON	Interoperabilidade ampla e payload legível.	Sem contrato claro de tipos, versões e campos opcionais.
Expor entidade interna como JSON	Pode acelerar protótipo controlado.	Acopla cliente ao modelo interno e dificulta evolução.
Acesso direto ao banco	Quase nunca é a melhor integração entre sistemas autônomos.	Quebra encapsulamento, segurança, governança e versionamento.

11. Pegadinhas FGV

- REST não é protocolo; normalmente usa HTTP, mas é estilo arquitetural.
- JSON não é REST, não é API e não é banco de dados; é formato de dados.
- HTTP é stateless; sessão, token e cookies podem existir, mas não mudam a natureza básica do protocolo.
- GET seguro não significa criptografado; seguro em HTTP significa que o método não pretende alterar estado do recurso.
- Idempotente não significa sem efeito; PUT e DELETE podem alterar estado, mas repetição tende ao mesmo efeito pretendido.
- POST não é automaticamente errado; ele é adequado para criação em coleção ou processamento não idempotente.
- PUT e PATCH não são sinônimos: PUT substitui representação; PATCH altera parcialmente, conforme contrato.
- Webhook não é fila nem mensageria por si só; é chamada HTTP disparada por evento.
- Webhook não elimina a necessidade de API de consulta; muitas vezes ele só notifica que algo ocorreu.
- API reversa não é API Gateway nem reverse proxy; no edital, leia próximo de webhook/callback.
- Acesso direto a banco de outro sistema pode parecer integração, mas cria acoplamento forte e risco de segurança.
- Status code errado pode denunciar alternativa: 202 não significa processado, 401 é autenticação, 403 é autorização.

12. Lista de questões autorais - padrão FGV

Resolva as questões sem consultar o gabarito. As alternativas foram construídas com distratores plausíveis, explorando confusões entre integração, REST, HTTP, JSON, webhook, polling e API reversa.

Questão 1

Um tribunal disponibiliza uma funcionalidade para que sistemas externos consultem a situação de processos administrativos. Cada processo é identificado por número único, e a consulta não deve alterar o estado do recurso. Considerando uma API RESTful sobre HTTP, a alternativa mais adequada é:

- (A) POST /consultarProcesso, com o número do processo no corpo, pois toda consulta com JSON deve utilizar POST.
- (B) GET /executarConsultaProcessual/{numero}, pois a palavra executar explicita a ação do serviço.
- (C) GET /processos/{numero}, pois o recurso é identificado pela URI e a consulta usa método seguro.
- (D) DELETE /processos/{numero}?consulta=true, pois DELETE também pode retornar a representação atual.
- (E) PUT /processos, com o número no header, pois PUT é o método padrão para obter recursos idempotentes.

Questão 2

Em uma integração corporativa, a equipe propõe que um sistema externo consulte diretamente tabelas de produção do sistema processual para reduzir a latência da integração. À luz de boas práticas de integração de sistemas, a crítica mais adequada é que essa solução:

- (A) é preferível a APIs, porque elimina qualquer dependência de contrato entre sistemas.
- (B) aumenta o acoplamento ao esquema interno, amplia riscos de segurança e dificulta evolução do sistema provedor.
- (C) é obrigatória em arquiteturas RESTful, pois REST exige acesso direto aos dados persistidos.
- (D) transforma automaticamente a integração em orientada a eventos, pois o banco passa a publicar alterações.
- (E) resolve o versionamento, porque alterações em tabelas são invisíveis para consumidores externos.

Questão 3

Sobre JSON - JavaScript Object Notation - em APIs de integração, assinale a afirmativa correta.

- (A) JSON é um protocolo de transporte que substitui o HTTP em APIs modernas.
- (B) JSON exige que toda API seja RESTful e stateless.
- (C) JSON é equivalente a XML Schema e define autenticação, autorização e versionamento.
- (D) JSON é um banco de dados documental usado diretamente como camada de persistência.
- (E) JSON é um formato textual e independente de linguagem para representar dados estruturados, como objetos e arrays.

Questão 4

Considere as afirmativas a seguir sobre métodos HTTP em APIs RESTful. I. GET é utilizado para recuperar representações e deve ser tratado como método seguro. II. PUT tende a ser idempotente quando usado para substituir o recurso identificado pela URI. III. POST é sempre idempotente, pois qualquer chamada repetida deve produzir o mesmo efeito no servidor. Está correto o que se afirma em:

- (A) I, II e III.
- (B) I e II, apenas.
- (C) I e III, apenas.
- (D) II e III, apenas.
- (E) III, apenas.

Questão 5

Um órgão público deseja notificar sistemas parceiros sempre que um processo for movimentado. Os parceiros não querem consultar periodicamente a API para descobrir novidades. A alternativa mais aderente ao cenário é:

- (A) exigir que os parceiros façam polling a cada segundo, pois isso reduz tráfego e elimina acoplamento temporal.
- (B) permitir acesso de leitura direta ao banco para que cada parceiro detecte alterações por conta própria.
- (C) substituir todos os endpoints REST por WSDL, pois notificação de eventos só ocorre via SOAP.
- (D) implementar webhooks/API reversa, com endpoint exposto pelo parceiro para receber chamadas quando o evento ocorrer.
- (E) usar apenas GET /processos para todas as operações, pois GET é o único método compatível com eventos.

Questão 6

Em uma API RESTful, após a criação de um novo recurso por POST /documentos, o status code mais específico para indicar criação bem-sucedida, em regra, é:

- (A) 201 Created.
- (B) 204 No Content, obrigatoriamente com corpo JSON.
- (C) 304 Not Modified.

- (D) 401 Unauthorized.
- (E) 503 Service Unavailable.

Questão 7

Um webhook de pagamento pode ser reenviado pelo provedor quando não recebe confirmação adequada. Para evitar que o mesmo evento seja processado duas vezes pelo consumidor, o desenho da integração deve priorizar:

- (A) uso exclusivo de XML, pois XML impede duplicidade de mensagens.
- (B) remoção de autenticação, pois autenticação aumenta latência e causa reenvios.
- (C) idempotência baseada em identificador único do evento ou chave de negócio.
- (D) uso de GET com corpo JSON, pois GET nunca é reenviado por provedores.
- (E) troca do HTTP por JSON, pois JSON controla transações distribuídas.

Questão 8

Assinale a opção que melhor distingue webhook de polling.

- (A) Webhook exige que o consumidor consulte periodicamente o provedor; polling dispensa chamadas de rede.
- (B) Webhook é sempre síncrono e bloqueante; polling é sempre assíncrono e orientado a eventos.
- (C) Webhook é um formato de payload; polling é um protocolo de autenticação.
- (D) Webhook só pode ser implementado com SOAP; polling só pode ser implementado com REST.
- (E) Webhook envolve notificação enviada pelo provedor ao consumidor quando ocorre evento; polling envolve consulta periódica feita pelo consumidor.

Questão 9

No contexto do edital, a expressão API reversa é mais bem compreendida como:

- (A) um banco de dados NoSQL que armazena respostas HTTP em ordem inversa.
- (B) um padrão em que o consumidor/parceiro expõe endpoint para ser chamado pelo provedor, usualmente em fluxo de callback ou webhook.
- (C) um algoritmo de criptografia que inverte payloads JSON para proteger dados pessoais.
- (D) um sinônimo técnico obrigatório de API Gateway, responsável por roteamento e rate limiting.
- (E) um tipo de REST que exige métodos HTTP usados ao contrário, como DELETE para criar e GET para remover.

Questão 10

Uma API retorna 401 Unauthorized quando o usuário está autenticado, mas não possui perfil para acessar determinado recurso. Tecnicamente, a resposta mais adequada para esse caso seria:

- (A) 200 OK, pois o usuário existe no sistema.
- (B) 201 Created, pois o acesso foi criado na sessão.
- (C) 404 Not Found, obrigatoriamente, pois toda falha de permissão deve ocultar o recurso.
- (D) 403 Forbidden, pois há autenticação, mas falta autorização para o recurso.
- (E) 503 Service Unavailable, pois a regra de permissão é indisponibilidade de negócio.

Questão 11

Em um desenho RESTful, a recomendação de usar nomes de recursos nas URIs, em vez de verbos de implementação, decorre principalmente da ideia de:

- (A) interface uniforme, em que os métodos HTTP expressam a ação e a URI identifica o recurso.
- (B) substituição do HTTP por JSON, reduzindo o número de endpoints.
- (C) obrigatoriedade de que toda API REST possua apenas um endpoint.
- (D) uso de WSDL para gerar contratos em tempo de compilação.
- (E) dependência direta entre cada endpoint e uma tabela física do banco.

Questão 12

Analise as afirmações sobre integração por webhooks. I. O consumidor deve validar origem, assinatura ou credencial antes de processar a mensagem. II. O consumidor deve responder rapidamente e pode encaminhar processamento pesado para fila ou rotina interna. III. O consumidor pode presumir que cada evento chegará uma única vez, dispensando controle de duplicidade. Está correto o que se afirma em:

- (A) I, II e III.
- (B) I e III, apenas.
- (C) II e III, apenas.
- (D) III, apenas.
- (E) I e II, apenas.

Questão 13

Relacione os conceitos aos seus significados. 1. Endpoint. 2. Payload. 3. Header HTTP. 4. Status code. A associação correta é:

- (A) 1-carga útil; 2-código de resultado; 3-recurso persistido; 4-ponto de acesso.
- (B) 1-protocolo criptográfico; 2-ponto de acesso; 3-banco documental; 4-carga útil.
- (C) 1-ponto de acesso da API; 2-carga útil; 3-metadado da mensagem; 4-código de resultado da resposta.
- (D) 1-método de autenticação; 2-URI física do banco; 3-código de erro; 4-formato XML.
- (E) 1-fila de mensagens; 2-proxy reverso; 3-evento de domínio; 4-entidade JPA.

Questão 14

Um sistema externo envia PATCH /usuarios/10 com payload contendo apenas o campo email. A semântica pretendida é atualizar parcialmente o recurso usuário. A afirmação correta é:

- (A) A operação é necessariamente segura, pois altera apenas um campo.
- (B) PATCH e PUT são sempre equivalentes e ambos sempre substituem toda a representação.
- (C) O endpoint deveria obrigatoriamente ser GET /alterarEmail, pois toda alteração parcial usa GET.
- (D) PATCH pode ser adequado para alteração parcial, desde que o contrato defina como aplicar o payload.
- (E) A operação deixa de ser HTTP porque payload JSON não pode ser enviado em requisições de atualização.

Questão 15

Uma equipe deseja expor a entidade JPA Processo diretamente como JSON em uma API pública, incluindo campos internos, relacionamentos preguiçosos e nomes de colunas. O principal risco arquitetural é:

- (A) impedir que a API use HTTPS, pois entidades JPA não podem ser trafegadas com TLS.
- (B) acoplar consumidores ao modelo interno de persistência, ampliando risco de quebra, vazamento e dificuldade de evolução.
- (C) tornar a API automaticamente SOAP, pois JPA sempre serializa XML.
- (D) eliminar a necessidade de autenticação, pois DTOs só são usados em sistemas privados.
- (E) impedir qualquer uso de JSON, pois JSON não aceita objetos aninhados.

Questão 16

Considere o diagrama textual: Angular -> API Quarkus -> API externa de protocolo -> resposta JSON. A API Quarkus transforma o payload interno em contrato público, chama o serviço externo e traduz o retorno. Esse componente intermediário está exercendo principalmente o papel de:

- (A) banco de dados orientado a grafos.
- (B) servidor DNS autoritativo.
- (C) adaptador/orquestrador de integração, isolando contratos e detalhes externos dos clientes.
- (D) webhook, pois toda chamada entre APIs é automaticamente push por evento.
- (E) arquivo batch, pois JSON só é processado em lote.

Questão 17

Assinale a alternativa incorreta sobre REST, HTTP e JSON.

- (A) Toda API que usa JSON é, necessariamente, RESTful.
- (B) HTTP define métodos, status codes, headers e modelo request/response.
- (C) REST pode ser implementado sobre HTTP explorando recursos, representações e interface uniforme.
- (D) JSON pode representar objetos, arrays, strings, números, booleanos e null.
- (E) Uma API REST pode usar status codes para expressar resultado técnico da requisição.

Questão 18

Um endpoint recebe POST /lotes/importacao e retorna 202 Accepted, com um identificador para consulta posterior. A interpretação mais adequada é:

- (A) o recurso foi processado por completo e o resultado final está garantido no corpo da resposta.
- (B) a requisição falhou por falta de autenticação.
- (C) o cliente tentou acessar recurso inexistente.
- (D) o servidor recusou a requisição por conflito de versão.
- (E) a solicitação foi aceita para processamento, possivelmente assíncrono, sem garantir conclusão imediata.

Questão 19

Sobre versionamento e evolução de contratos de API, assinale a afirmativa mais adequada.

- (A) Remover campos de uma resposta pública é sempre seguro, pois clientes REST ignoram automaticamente qualquer mudança.
- (B) Alterar significado de campo mantendo o mesmo nome nunca afeta consumidores se o payload continuar em JSON.

- (C) Versionamento é desnecessário quando se usa HTTPS, pois TLS preserva compatibilidade semântica.
- (D) Mudanças aditivas e compatíveis tendem a ser menos disruptivas; mudanças que removem ou renomeiam campos exigem maior cuidado e, muitas vezes, nova versão.
- (E) A versão da API deve ser sempre igual à versão do banco de dados relacional subjacente.

Questão 20

Um sistema legado só consegue exportar diariamente um arquivo CSV com movimentações. O TJSC deseja disponibilizar esses dados a sistemas modernos via API JSON, sem expor o formato legado aos consumidores. A solução mais adequada é:

- (A) obrigar todos os consumidores a lerem diretamente o CSV legado, pois isso preserva baixo acoplamento.
- (B) criar uma camada adaptadora que importe/valide o arquivo e exponha contrato moderno por API, isolando o legado dos consumidores.
- (C) substituir JSON por DNS, pois DNS é o protocolo adequado para transformar arquivos em APIs.
- (D) usar DELETE para consultar movimentações, pois sistemas legados não suportam GET.
- (E) eliminar contratos, pois sistemas em lote não precisam de validação de dados.

13. Gabarito resumido

1-C	2-B	3-E	4-B	5-D
6-A	7-C	8-E	9-B	10-D
11-A	12-E	13-C	14-D	15-B
16-C	17-A	18-E	19-D	20-B

Distribuição orgânica: A = 3; B = 5; C = 4; D = 4; E = 4. Não há rodízio regular nem sequência simétrica de gabarito.

14. Comentários completos

Questão 1 - Gabarito: C (GET /processos/{numero})

- **A.** Tentadora porque usa POST com JSON, algo comum em sistemas corporativos. Mas consulta segura de recurso identificado não deve usar POST apenas por carregar JSON.
- **B.** Parece clara por nomear a ação, mas REST tende a modelar recursos com substantivos. O método GET já expressa a ação de consulta.
- **C.** Correta. A URI identifica o recurso processo e GET é método seguro para recuperar representação sem alterar estado.
- **D.** Errada. DELETE indica remoção do recurso, não consulta. Query parameter não muda a semântica do método.
- **E.** Errada. PUT é voltado a substituir/criar recurso identificado, não a recuperar representação.

Questão 2 - Gabarito: B (acoplamento ao schema interno)

- **A.** Parece reduzir camadas, mas não elimina contrato; cria contrato implícito e frágil com tabelas internas.
- **B.** Correta. Acesso direto ao banco acopla consumidores ao schema, dificulta evolução, amplia superfície de acesso e quebra encapsulamento.
- **C.** Errada. REST não exige acesso direto à persistência; ao contrário, API deve abstrair implementação interna.
- **D.** Errada. Consultar banco não torna a integração orientada a eventos. Evento pressupõe publicação/notificação de ocorrência.
- **E.** Errada. Alterações em tabelas são altamente visíveis para quem consulta diretamente o banco.

Questão 3 - Gabarito: E (formato textual de dados)

- **A.** Errada. JSON não transporta mensagens sozinho; normalmente é payload sobre HTTP ou outro meio.
- **B.** Errada. JSON não torna uma API RESTful nem define statelessness.
- **C.** Errada. JSON não define autenticação/autorização/versionamento; pode ser combinado com schemas e documentação.
- **D.** Errada. Existem bancos documentais que armazenam documentos semelhantes a JSON/BSON, mas JSON em si é formato de dados.
- **E.** Correta. JSON é formato leve, textual e independente de linguagem para dados estruturados, com objetos e arrays.

Questão 4 - Gabarito: B (I e II)

- **A.** Errada porque inclui III. POST não é sempre idempotente.
- **B.** Correta. GET é seguro; PUT, quando usado para substituir o recurso identificado, tende a ser idempotente.
- **C.** Errada porque III é falsa.
- **D.** Errada porque III é falsa e I também é correta.
- **E.** Errada. III isolada é falsa.

Questão 5 - Gabarito: D (webhook/API reversa)

- **A.** Polling a cada segundo pode aumentar tráfego e carga; não reduz naturalmente acoplamento nem garante eficiência.
- **B.** Acesso direto ao banco é frágil e acoplado.
- **C.** SOAP/WSDL pode existir em serviços web, mas notificação de eventos não exige SOAP.
- **D.** Correta. Webhook/API reversa permite que o parceiro exponha endpoint e receba notificação quando o evento ocorrer.
- **E.** GET não representa eventos por si só; e operações de notificação não são resolvidas com GET único para tudo.

Questão 6 - Gabarito: A (201 Created)

- **A.** Correta. 201 Created é o código típico para criação bem-sucedida de recurso.
- **B.** 204 indica sucesso sem corpo, mas não é o código mais específico para criação; além disso não tem corpo obrigatório.
- **C.** 304 está ligado a cache/condicional, não criação.

- D. 401 trata autenticação ausente/inválida, não sucesso.
- E. 503 indica indisponibilidade temporária.

Questão 7 - Gabarito: C (idempotência)

- A. XML não impede duplicidade; formato não resolve semântica de processamento.
- B. Remover autenticação piora segurança e não resolve retry.
- C. Correta. Identificador único do evento ou chave de negócio permite reconhecer duplicata e evitar duplo processamento.
- D. GET com corpo não é solução adequada; provedores podem reenviar chamadas independentemente do método.
- E. JSON não controla transações distribuídas nem elimina duplicidade.

Questão 8 - Gabarito: E (push x pull)

- A. Inverte os conceitos. Polling é que consulta periodicamente.
- B. Webhook pode ser assíncrono por evento; polling não é sempre assíncrono.
- C. Webhook é mecanismo de chamada/evento, não formato de payload.
- D. Ambos podem usar HTTP; não há exigência de SOAP para webhook.
- E. Correta. Webhook é push por evento; polling é pull periódico.

Questão 9 - Gabarito: B (callback/endpoint do consumidor)

- A. Não tem relação com banco NoSQL.
- B. Correta. API reversa, no contexto de webhooks/callbacks, inverte o sentido tradicional: consumidor expõe endpoint e provedor chama.
- C. Não é algoritmo de criptografia.
- D. Plausível pela expressão reverse, mas API Gateway/reverse proxy é outro conceito.
- E. Errada. Métodos HTTP não são usados ao contrário.

Questão 10 - Gabarito: D (403 Forbidden)

- A. 200 indicaria sucesso, inadequado para negação de acesso.
- B. 201 é criação de recurso, não decisão de autorização.
- C. 404 pode ser usado estrategicamente para ocultar recurso em alguns contratos, mas não é a resposta tecnicamente mais direta para autenticado sem permissão.
- D. Correta. 403 indica que o servidor entendeu e o cliente está autenticado, mas não autorizado.
- E. 503 indica indisponibilidade, não falta de perfil.

Questão 11 - Gabarito: A (interface uniforme)

- A. Correta. Em REST, URI identifica recurso e método HTTP expressa a operação, compondo interface uniforme.
- B. REST não substitui HTTP por JSON.
- C. REST não exige endpoint único.
- D. WSDL é típico de SOAP e não é base de REST.
- E. API não deve necessariamente espelhar tabela física; isso gera acoplamento.

Questão 12 - Gabarito: E (I e II)

- A. Errada porque III é falsa. Webhooks podem chegar duplicados.
- B. Errada porque II é correta e III é falsa.
- C. Errada porque III é falsa e I é correta.
- D. Errada. III é a armadilha clássica: retry e duplicidade são esperados.
- E. Correta. Deve-se validar origem e responder rapidamente, encaminhando processamento pesado se necessário.

Questão 13 - Gabarito: C (associação correta)

- A. Troca os conceitos: payload não é endpoint; status code não é recurso persistido.
- B. Mistura TLS, endpoint e banco documental sem aderência.
- C. Correta. Endpoint é ponto de acesso; payload é carga útil; header leva metadados; status code indica resultado.
- D. Confunde endpoint com autenticação, payload com banco e status com XML.
- E. Traz termos reais de outros tópicos, mas fora do significado pedido.

Questão 14 - Gabarito: D (PATCH parcial)

- A. Alterar email muda estado; não é método seguro.
- B. PUT e PATCH não são equivalentes: PUT substitui; PATCH altera parcialmente conforme contrato.
- C. GET não deve alterar estado; verbo na URI é menos aderente a REST.
- D. Correta. PATCH é adequado para atualização parcial quando o contrato define semântica do payload.
- E. HTTP pode transportar payload JSON em métodos de atualização.

Questão 15 - Gabarito: B (acoplamento à persistência)

- A. TLS independe de JPA.
- B. Correta. Expor entidade interna acopla cliente ao modelo de persistência, vaza detalhes e dificulta mudanças.
- C. JPA não transforma API em SOAP.
- D. DTO não é mecanismo de autenticação; autenticação continua necessária conforme contexto.
- E. JSON aceita objetos aninhados e arrays.

Questão 16 - Gabarito: C (adaptador/orquestrador)

- A. Não há grafo no cenário.
- B. DNS resolve nomes, não traduz contratos de API.
- C. Correta. A API Quarkus isola cliente de contratos externos e transforma payloads, papel típico de adaptação/orquestração de integração.
- D. Chamada entre APIs não é automaticamente webhook; webhook depende de evento e sentido da chamada.
- E. JSON não é exclusivo de processamento em lote.

Questão 17 - Gabarito: A (incorreta)

- A. Correta como resposta da questão, pois é a alternativa incorreta. Usar JSON não garante RESTful.
- B. Verdadeira. HTTP inclui métodos, status codes, headers e request/response.
- C. Verdadeira. REST sobre HTTP explora recursos, representações e interface uniforme.
- D. Verdadeira. Esses são tipos e estruturas de JSON.
- E. Verdadeira. Status codes fazem parte do contrato técnico da resposta HTTP.

Questão 18 - Gabarito: E (202 Accepted)

- A. Errada. 202 não garante conclusão imediata.
- B. Autenticação ausente/inválida seria 401.
- C. Recurso inexistente seria 404.
- D. Conflito de versão/estado tenderia a 409.
- E. Correta. 202 indica aceitação para processamento, possivelmente assíncrono.

Questão 19 - Gabarito: D (versionamento e compatibilidade)

- A. Remover campos pode quebrar clientes que dependem deles.
- B. Manter nome e alterar significado pode ser ainda mais perigoso, pois quebra semântica silenciosamente.
- C. TLS protege transporte; não resolve compatibilidade semântica de contrato.
- D. Correta. Mudanças aditivas tendem a ser compatíveis; remoção/renomeação exige cuidado e possivelmente nova versão.
- E. Versão de API e versão de banco são preocupações diferentes.

Questão 20 - Gabarito: B (camada adaptadora)

- A. Expor CSV legado a todos aumenta acoplamento e espalha detalhes internos.
- B. Correta. Adaptador isola legado, valida dados e oferece contrato moderno aos consumidores.
- C. DNS não transforma arquivos em APIs.
- D. DELETE não é método de consulta.
- E. Mesmo lote precisa de contrato, validação e governança de dados.

15. Referências técnicas consultadas

- FGV Conhecimento - Edital TJSC 2026, Analista de Sistemas, conteúdo de Arquitetura de Sistemas e Integração.
- Manual Mestre de Calibração FGV para Aulas e Questões - Projeto TJSC 2026.
- RFC 9110 - HTTP Semantics. Define HTTP como protocolo stateless de aplicação e consolida semântica de métodos, mensagens e URIs.
- RFC 8259 - The JavaScript Object Notation (JSON) Data Interchange Format. Define JSON como formato textual leve e independente de linguagem.
- RFC 9205 - Building Protocols with HTTP. Boas práticas para APIs e protocolos construídos sobre HTTP.
- Microsoft Learn - Web API Design Best Practices. Referência de desenho RESTful, recursos, métodos e convenções.
- OpenAPI Specification 3.2.0 - Webhooks. Define seção webhooks como requisições iniciadas fora de uma chamada direta da API.
- Swagger/OpenAPI Docs - Callbacks. Exemplo de callbacks/webhooks assíncronos fora do fluxo regular request-response.
- IBM Documentation - Webhooks. Descreve webhook como chamada HTTP originada do sistema provedor para sistema remoto configurado.
- FGV - prova correlata de TI: questão sobre API REST, métodos HTTP e statelessness usada apenas para calibrar estilo, sem reprodução literal.